

# uni-app中button和input表单组件的用法及注意事项

在 uni-app 中，`button`（按钮）和 `input`（输入框）是最核心的表单组件，适配多端（微信小程序、App、H5 等），以下是它们的**基础用法、核心属性、常见场景及注意事项**，结合示例代码讲解：

## 一、button 按钮组件

### 1. 核心特性

- 多端统一样式，支持自定义外观、类型、状态；
- 内置点击反馈、加载状态、开放能力（如获取用户信息、支付）；
- 支持 `open-type` 对接各平台原生能力（如微信小程序授权）。

### 2. 基础属性（常用）

| 属性名                    | 说明                                         | 可选值                                    | 默认值     |
|------------------------|--------------------------------------------|----------------------------------------|---------|
| <code>type</code>      | 按钮样式类型                                     | primary（主色）/default（默认）/warn（警告）       | default |
| <code>size</code>      | 按钮大小                                       | default / mini（迷你）                     | default |
| <code>plain</code>     | 是否镂空（透明背景，边框样式）                            | true / false                           | false   |
| <code>disabled</code>  | 是否禁用                                       | true / false                           | false   |
| <code>loading</code>   | 是否显示加载状态（加载中禁用点击）                          | true / false                           | false   |
| <code>form-type</code> | 表单提交/重置（配合 <code>&lt;form&gt;</code> 标签使用） | submit（提交）/reset（重置）                   | -       |
| <code>open-type</code> | 开放能力（微信小程序为主）                              | getUserInfo / getPhoneNumber / share 等 | -       |
| <code>@click</code>    | 点击事件（核心）                                   | 自定义事件处理函数                              | -       |

### 3. 示例代码

```
1 <template>  
code block  
2   <view class="button-demo">  
3     <!-- 基础按钮 -->  
4     <button type="primary" @click="handleClick">主色调按钮</button>  
5     <!-- 迷你镂空按钮 -->  
6     <button type="default" size="mini" plain @click="handleMiniClick">迷你镂空按钮</button>  
7     <!-- 禁用按钮 -->  
8     <button type="warn" disabled>禁用警告按钮</button>  
9     <!-- 加载状态按钮 -->  
10    <button type="primary" :loading="isLoading" @click="handleLoading">  
11      {{ isLoading ? "加载中..." : "点击加载" }}  
12    </button>  
13    <!-- 表单提交按钮（配合form） -->  
14    <form @submit="formSubmit">  
15      <button form-type="submit">提交表单</button>  
16      <button form-type="reset">重置表单</button>  
17    </form>  
18    <!-- 微信小程序授权按钮（open-type） -->  
19    <button open-type="getUserInfo" @getuserinfo="getUserInfo">获取微信用户信息</button>  
20  </view>  
21 </template>  
22  
23 <script setup>  
24 import { ref } from "vue";  
25 const isLoading = ref(false);  
26  
27 // 基础点击事件  
28 const handleClick = () => {  
29   uni.showToast({ title: "主按钮被点击", icon: "none" });  
30 };  
31  
32 // 迷你按钮点击  
33 const handleMiniClick = () => {  
34   uni.showToast({ title: "迷你按钮点击", icon: "none" });  
35 };  
36  
37 // 加载状态切换  
38 const handleLoading = () => {  
39   isLoading.value = true;  
40   // 模拟异步操作  
41   setTimeout(() => {  
42     isLoading.value = false;  
43   }, 2000);  
44 };  
45
```

```
46 // 表单提交
47 const formSubmit = (e) => {
48   console.log("表单提交数据: ", e.detail.value);
49 };
50
51 // 微信用户信息授权（仅小程序生效）
52 const getUserInfo = (e) => {
53   console.log("用户信息: ", e.detail.userInfo);
54 };
55 </script>
56
57 <style scoped>
58   .button-demo {
59     padding: 20rpx;
60     display: flex;
61     flex-direction: column;
62     gap: 20rpx;
63   }
64 </style>
```

## 二、input 输入框组件

### 1. 核心特性

- 支持文本、数字、密码、手机号等输入类型；
- 适配多端键盘类型（如数字键盘、身份证键盘）；
- 支持输入过滤、焦点控制、字数限制；
- 绑定 `v-model` 实现双向数据绑定（uni-app 兼容 Vue 语法）。

### 2. 基础属性（常用）

| 属性名                        | 说明                   | 可选值                                       | 默认值   |
|----------------------------|----------------------|-------------------------------------------|-------|
| <code>type</code>          | 输入类型                 | text / number / idcard / digit / password | text  |
| <code>value/v-model</code> | 输入框内容（双向绑定用 v-model） | 字符串                                       | -     |
| <code>placeholder</code>   | 占位提示文字               | 字符串                                       | -     |
| <code>disabled</code>      | 是否禁用                 | true / false                              | false |
| <code>maxlength</code>     | 最大输入长度（-1 表示无限制）     | 数字                                        | -1    |

|                |                               |                                  |       |
|----------------|-------------------------------|----------------------------------|-------|
| focus          | 是否自动获取焦点                      | true / false                     | false |
| confirm-type   | 键盘右下角按钮文字（多端适配）               | done / go / next / search / send | done  |
| password       | 是否密码类型（优先级低于 type="password"） | true / false                     | false |
| @input         | 输入内容实时触发                      | 自定义函数（参数为输入内容）                   | -     |
| @confirm       | 点击键盘完成按钮触发                    | 自定义函数                            | -     |
| @focus / @blur | 获取/失去焦点触发                     | 自定义函数                            | -     |

### 3. 示例代码

Code block

```
1  <template>
2    <view class="input-demo">
3      <!-- 基础文本输入框 -->
4      <input
5        v-model="username"
6        type="text"
7        placeholder="请输入用户名"
8        @input="handleInput"
9      />
10
11     <!-- 密码输入框 -->
12     <input
13       v-model="password"
14       type="password"
15       placeholder="请输入密码"
16       maxLength="16"
17     />
18
19     <!-- 数字输入框（仅允许数字） -->
20     <input
21       v-model="phone"
22       type="number"
23       placeholder="请输入手机号"
24       maxLength="11"
25       @confirm="handleConfirm"
26     />
27
28     <!-- 禁用输入框 -->
```

```
29     <input
30       value="禁用状态"
31       disabled
32       placeholder="禁用后无法输入"
33     />
34
35     <!-- 带清除按钮的输入框（结合uni内置样式） -->
36     <view class="input-clear">
37       <input v-model="searchText" placeholder="请输入搜索内容" />
38       <button size="mini" plain @click="searchText = ''" v-if="searchText">清除
39     </button>
39     </view>
40   </view>
41 </template>
42
43 <script setup>
44 import { ref } from "vue";
45 // 双向绑定数据
46 const username = ref("");
47 const password = ref("");
48 const phone = ref("");
49 const searchText = ref("");
50
51 // 实时输入监听
52 const handleInput = (e) => {
53   console.log("实时输入内容: ", e.detail.value);
54 };
55
56 // 键盘完成按钮触发
57 const handleConfirm = () => {
58   uni.showToast({ title: `手机号: ${phone.value}`, icon: "none" });
59 };
60 </script>
61
62 <style scoped>
63 .input-demo {
64   padding: 20rpx;
65   display: flex;
66   flex-direction: column;
67   gap: 30rpx;
68 }
69 input {
70   padding: 20rpx;
71   border: 1px solid #eee;
72   border-radius: 8rpx;
73 }
74 .input-clear {
```

```
75     display: flex;
76     align-items: center;
77     gap: 10rpx;
78   }
79 </style>
```

### 三、button + input 组合场景（表单提交）

最常见的登录/注册表单示例，结合数据校验：

Code block

```
1  <template>
2    <view class="form-login">
3      <input
4        v-model="phone"
5        type="number"
6        placeholder="请输入手机号"
7        maxlength="11"
8      />
9      <input
10       v-model="code"
11       type="number"
12       placeholder="请输入验证码"
13       maxlength="6"
14     />
15     <!-- 倒计时按钮 -->
16     <button
17       type="default"
18       size="mini"
19       :disabled="countDown > 0"
20       @click="getCode"
21     >
22       {{ countDown > 0 ? `${countDown}s后重新获取` : "获取验证码" }}
23     </button>
24     <!-- 提交按钮 -->
25     <button
26       type="primary"
27       @click="submitForm"
28       :loading="isSubmitting"
29     >
30       登录
31     </button>
32   </view>
33 </template>
34
35 <script setup>
```

```
36 import { ref } from "vue";
37 const phone = ref("");
38 const code = ref("");
39 const countdown = ref(0);
40 const isSubmitting = ref(false);
41
42 // 获取验证码
43 const getCode = () => {
44   // 手机号校验
45   if (!/^1[3-9]\d{9}$/.test(phone.value)) {
46     uni.showToast({ title: "请输入正确手机号", icon: "none" });
47     return;
48   }
49   // 倒计时逻辑
50   countdown.value = 60;
51   const timer = setInterval(() => {
52     countdown.value--;
53     if (countdown.value <= 0) {
54       clearInterval(timer);
55     }
56   }, 1000);
57   // 模拟发送验证码请求
58   uni.showToast({ title: "验证码已发送", icon: "none" });
59 };
60
61 // 表单提交
62 const submitForm = () => {
63   if (!phone.value || !code.value) {
64     uni.showToast({ title: "请填写完整信息", icon: "none" });
65     return;
66   }
67   isSubmitting.value = true;
68   // 模拟接口请求
69   setTimeout(() => {
70     isSubmitting.value = false;
71     uni.showToast({ title: "登录成功", icon: "success" });
72   }, 1500);
73 };
74 </script>
75
76 <style scoped>
77 .form-login {
78   padding: 40rpx 20rpx;
79   display: flex;
80   flex-direction: column;
81   gap: 30rpx;
82 }
```

```
83   input {
84     padding: 20rpx;
85     border: 1px solid #e5e5e5;
86     border-radius: 8rpx;
87   }
88 </style>
```

## 四、关键注意事项

### 1. 多端适配：

- `input` 的 `type="number"` 在 H5 端可能显示数字键盘，但部分浏览器仍允许输入非数字，需手动校验；
- `button` 的 `open-type` 仅对应平台生效（如 `getUserInfo` 仅微信小程序可用），其他端无效果。

### 2. 双向绑定：

- uni-app 中 `v-model` 完全兼容 `input`，底层映射为 `value` + `@input` 事件，无需手动处理。

### 3. 样式定制：

- 原生组件样式可通过 `::v-deep` (Vue2) / `:deep()` (Vue3) 穿透修改，或自定义 class 覆盖；
- 避免直接修改 `uni-button` / `uni-input` 全局样式，防止多端样式冲突。

### 4. 性能优化：

- 避免在 `@input` 事件中执行复杂逻辑（如频繁请求接口），可加防抖处理；
- 禁用状态的按钮/输入框，需设置 `disabled` 而非仅隐藏，保证多端交互一致性。

## 五、拓展能力

- `input` 支持 `confirm-hold`（点击完成按钮不收起键盘）、`auto-height`（多行文本自适应高度，需 `type="textare"`）；
- `button` 支持 `hover-class`（点击态样式）、`lang`（指定返回用户信息的语言，微信小程序）；
- 结合 `uni-forms` 组件（uni-app 扩展组件）可实现更复杂的表单校验、联动等功能。

（注：文档部分内容可能由 AI 生成）